

A few questions before...

Why did you choose this course?

What do you expect?

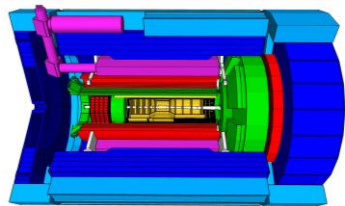
...

Do you have a notebook?

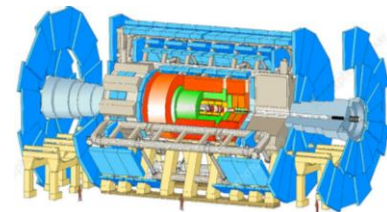
Which operating system is installed on your PC?

What does *operating system* (**OS**) mean?

What is a script and what is a bash shell?



Introduction



Through the analysis of data taken in high energy physics experiments we (we physicists...) try to determine the properties of elementary particles and the probabilities of occurrence of natural phenomena only visible at the high energy scale. We also search for new phenomena such as the production of new particles (predicted or not by a given theory) or anomalous cross sections or distributions. These measurements can also be used to extract or fit fundamental constants within the context of a given theoretical model, thereby providing the experimental inputs required to address phenomenology of elementary particles.

My **work plan** includes (24h lessons (L) and 36h lab (E) and introduces to some necessary steps for data analysis in hep and beyond.

1) The **first part** concerns useful software bases for HEP: an introduction to: Unix, shell scripts, makefiles etc. and **ROOT tools**

2) The second, **ATLAS** (today and tomorrow) and **ePIC** (tomorrow) experiments:

LHC-ATLAS apparatus and goals

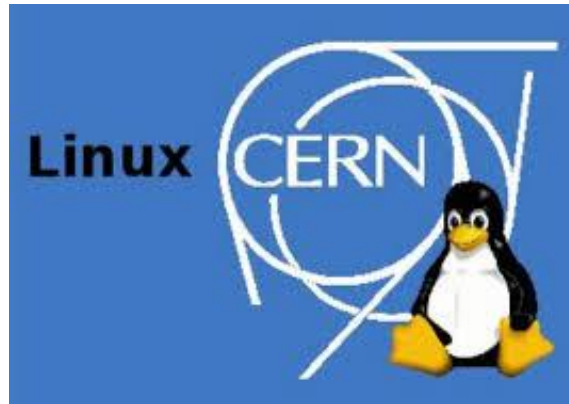
EIC-EPIC apparatus and goals

3) Monte Carlo uses in HEP (*generator and simulation of apparatus*)

4) Fitting methods

At the end of the course: the final exam. **The final vote take into account the discussions and activities during the course.**

We will work on unix-like machines because it is the most used operating system in **hep** (*high energy physics*).



Mac PCs have a unix-like environment, other pc use linux or linux virtual machine under windows.

For all (except Mac users)... I suggest **UBUNTU** OS, it is an open-source Linux OS.

Note1: An **operating system (OS)** is system software that manages computer hardware, software resources, and provides common services for computer programs.

Example: Microsoft **Windows**, Apple's **macOS** and iOS, Google's **Android** and **Chrome OS**, and open-source systems like **Linux** (including distributions such as **Ubuntu**) and **Unix**.

The **Unix** operating system was conceived and implemented in 1969 at AT&T's Bell Laboratories in USA by K. Thompson, D. Ritchie, D. McIlroy, and J. Ossanna. It was first released in 1971 and was initially entirely written *in assembly language* (a programming language that communicates with the hardware of a computer directly), a common practice at the time. Later, in 1973, Unix was re-written in the programming *language C* by Dennis Ritchie.

With a legal glitch forcing AT&T to license the operating system's source code to anyone who asked, Unix quickly grew and became widely adopted by academic institutions and businesses. In 1984, AT&T divested itself of Bell Labs. Free of the legal glitch requiring free licensing, Bell Labs began selling Unix as a proprietary product.

Note: *High-level programming languages* offer a higher level of abstraction, making it easier for developers to write code quickly and efficiently (C++, Python, etc).

Low-level programming languages offer a lower level of abstraction, which means that developers have to write more code to perform the same task (Assembly, etc..).

Shell is an Interpreter. A C++ compiler translates all the code before execution, while the interpreter (shell) translates it during execution, line by line.

Linux is a Unix-like computer operating system (OS) assembled under the model of free and open source software development and distribution (unlike Microsoft Windows, Unix, MacOS).

(Born in 1991, his father is Linus Benedict Torvalds)

Ubuntu is the most popular Linux distribution on desktop/laptop personal computers.

Ubuntu is a computer operating system based on the Debian **GNU/Linux** distribution and distributed as free and open source software, using its own desktop environment. It is named after the Southern African philosophy of ubuntu ("humanity towards others").

Note2: GNU ([/gh-nu:/](#) ([listen](#)) is an extensive collection of free software, which can be used as an operating system or can be used in parts with other operating systems.

The use of the completed GNU tools led to the family of operating systems popularly known as Linux.

Example: **Emacs and gcc.**



Note3: GUI (GOO-ee), **graphical user interface**, is a form of user interface that allows users to interact with electronic devices through graphical icons and audio indicator such as primary notation.

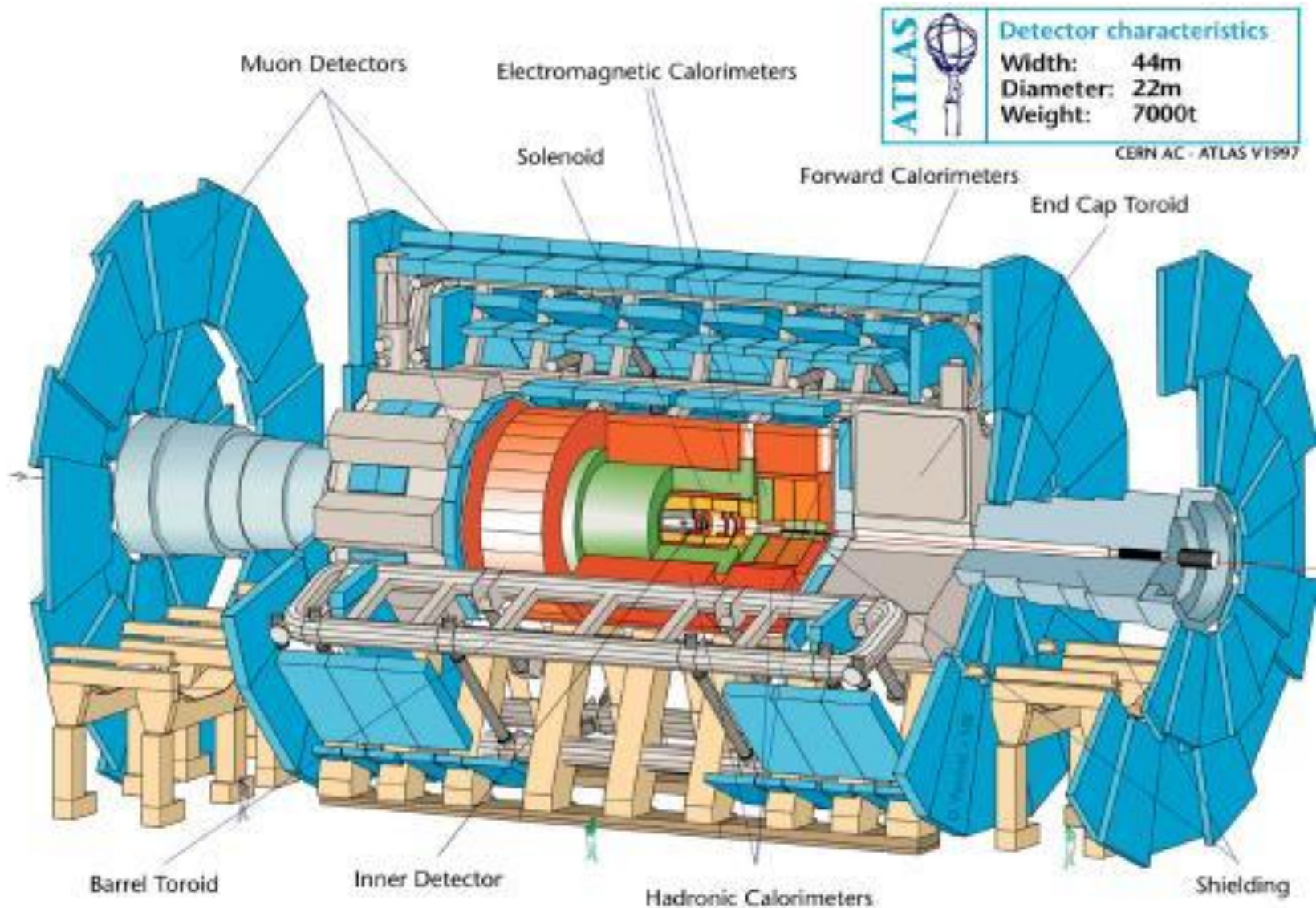
GUIs use visual elements like **windows**, **icons**, and **menus** to allow users to interact with software more intuitively than with text-based command-line interfaces

lun	mar	mer	gio	ven
21	22	23	24	25
10:30 - 11:30 QUANTUM FIELD THEORY II Lezione PHYSICS (2) FIS 4 A. PAPA	08:30 - 10:30 PARTICLE PHYSICS PHENOMENOLOGY I Lezione PHYSICS (2) FIS 4 S. FAZIO	08:30 - 09:30 PARTICLE PHYSICS PHENOMENOLOGY I Esercitazione PHYSICS (2) FIS 4 S. FAZIO	08:30 - 10:30 PARTICLE PHYSICS PHENOMENOLOGY II Esercitazione PHYSICS (2) FIS 4 E. MEONI	09:30 - 11:30 PARTICLE PHYSICS PHENOMENOLOGY II Lezione PHYSICS (2) D E. MEONI
	10:30 - 12:30 NUCLEAR AND PARTICLE PHYSICS LABORATORY II Lezione PHYSICS (2) FIS 4 M. CAPUA	09:30 - 11:30 NUCLEAR AND PARTICLE PHYSICS LABORATORY II Esercitazione PHYSICS (2) FIS 4 M. CAPUA	10:30 - 11:30 PARTICLE PHYSICS PHENOMENOLOGY I Esercitazione PHYSICS (2) FIS 4 S. FAZIO	
	14:30 - 16:30 STATISTICAL MECHANICS Esercitazione PHYSICS (2) FIS 4 M. ROSSI	11:30 - 12:30 STATISTICAL MECHANICS Lezione PHYSICS (2) FIS 4 D. GIULIANO	11:30 - 12:30 STATISTICAL MECHANICS Lezione PHYSICS (2) FIS 3 D. GIULIANO	
	16:30 - 17:30 QUANTUM FIELD THEORY II Lezione PHYSICS (2) FIS 4 A. PAPA	14:30 - 15:30 STATISTICAL MECHANICS Esercitazione PHYSICS (2) FIS 3 M. ROSSI	14:30 - 15:30 QUANTUM FIELD THEORY II Lezione PHYSICS (2) FIS 4 A. PAPA	
		15:30 - 17:30 QUANTUM FIELD THEORY II Esercitazione PHYSICS (2) FIS 4 L. DELLE ROSE	15:30 - 17:30 NUCLEAR AND PARTICLE PHYSICS LABORATORY II Esercitazione PHYSICS (2) FIS 4 M. CAPUA	

optional courses →

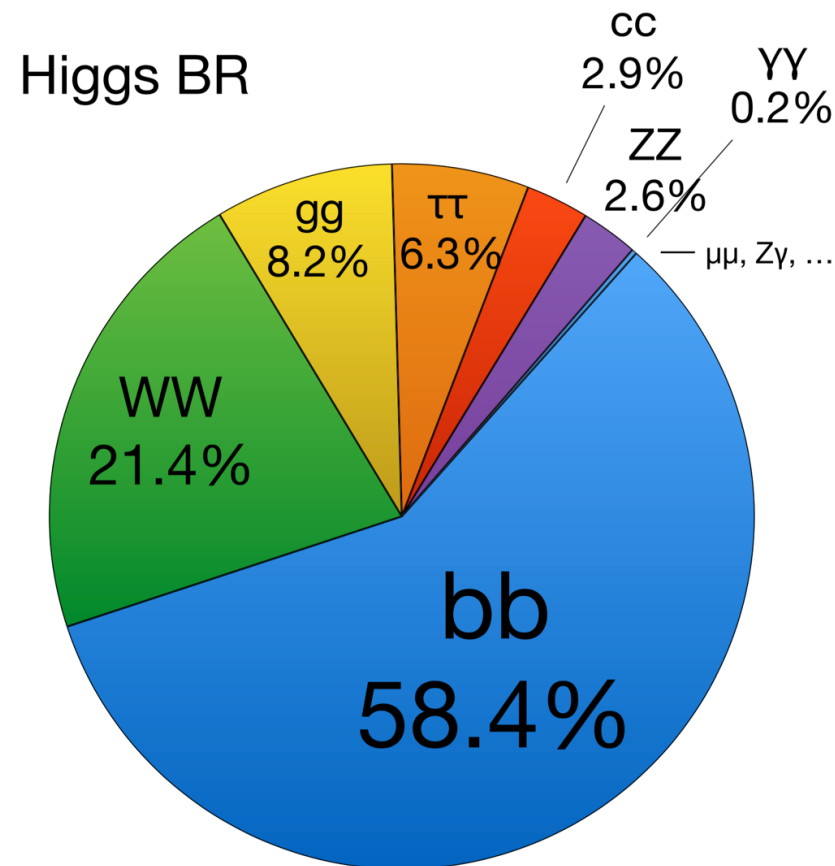
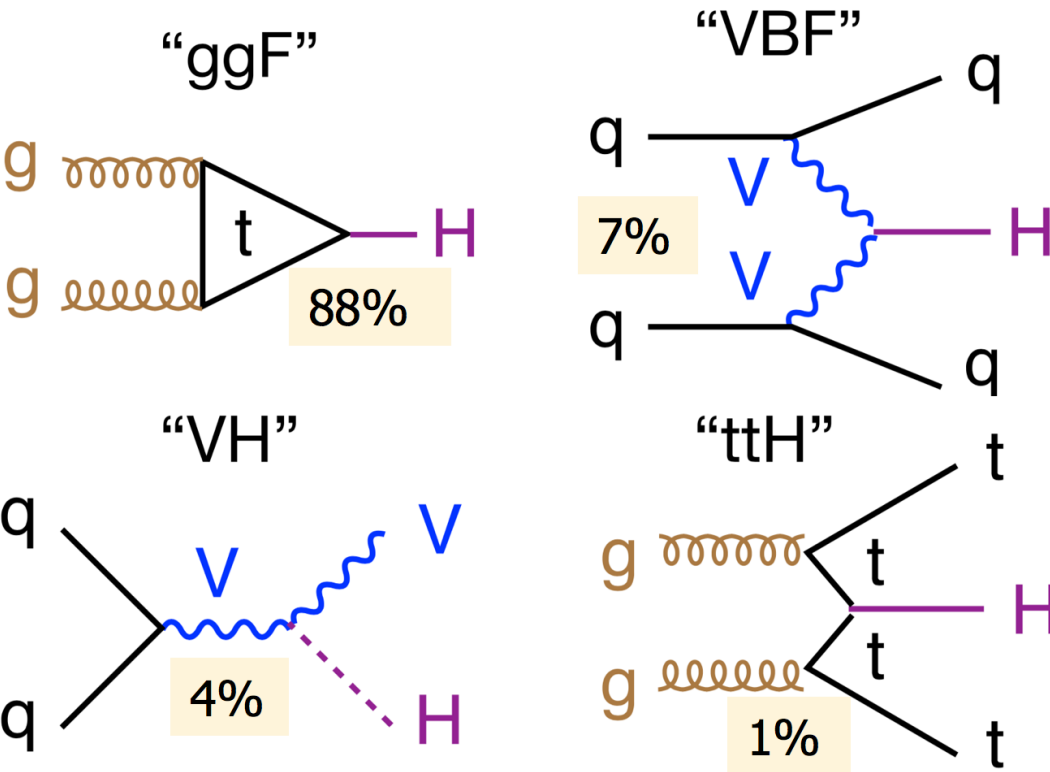
settembre 2026				
lun	mar	mer	gio	ven
21	22	23	24	25
11:30 - 13:30 HIGH-ENERGY ASTROPHYSICS Lezione PHYSICS (2) FIS 1 S. PERRI	10:30 - 12:30 NUCLEAR AND PARTICLE PHYSICS LABORATORY II Lezione PHYSICS (2) FIS 4 M. CAPUA	09:30 - 11:30 NUCLEAR AND PARTICLE PHYSICS LABORATORY II Esercitazione PHYSICS (2) FIS 4 M. CAPUA	08:30 - 10:30 MATHEMATICS EDUCATION I Lezione MATHEMATICS (2) LAB 30A A. SERPE	08:30 - 11:30 MESOPHASES AND METASTRUCTURES Lezione PHYSICS (2) FIS 4 V. CALIGIURI
	14:30 - 16:30 HIGH-ENERGY ASTROPHYSICS Esercitazione PHYSICS (2) FIS 3 G. PRETE		10:30 - 11:30 MATHEMATICS EDUCATION I Laboratorio MATHEMATICS (2) LAB 30A A. SERPE	08:30 - 09:30 HIGH-ENERGY ASTROPHYSICS Lezione PHYSICS (2) FIS 1 S. PERRI
	14:30 - 16:30 MATHEMATICS EDUCATION I Lezione MATHEMATICS (2) LAB 30A A. SERPE		14:30 - 16:30 COMPUTATIONAL BIOPHYSICS Lezione PHYSICS (2) FIS 3 B. RIZZUTI	09:30 - 11:30 COMPUTATIONAL BIOPHYSICS Esercitazione PHYSICS (2) FIS 3 B. RIZZUTI
	16:30 - 17:30 MATHEMATICS EDUCATION I Laboratorio MATHEMATICS (2) LAB 30A A. SERPE		15:30 - 17:30 NUCLEAR AND PARTICLE PHYSICS LABORATORY II Esercitazione PHYSICS (2) FIS 4 M. CAPUA	

ATLAS is a perfect example of particle physics detector

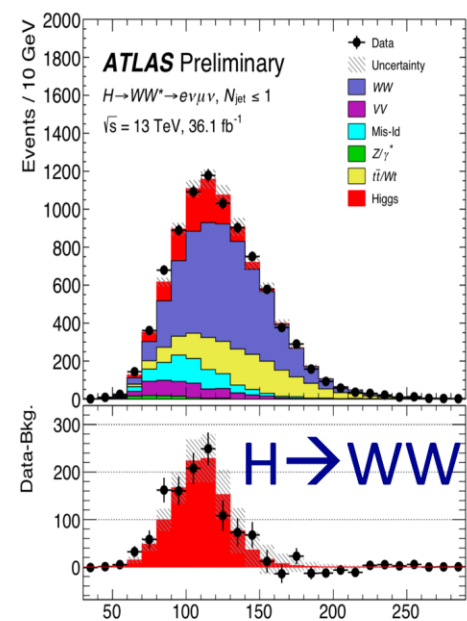
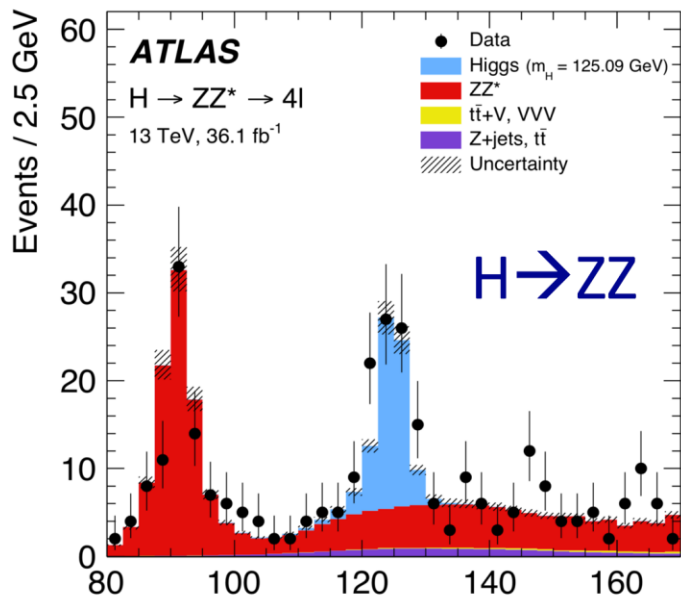
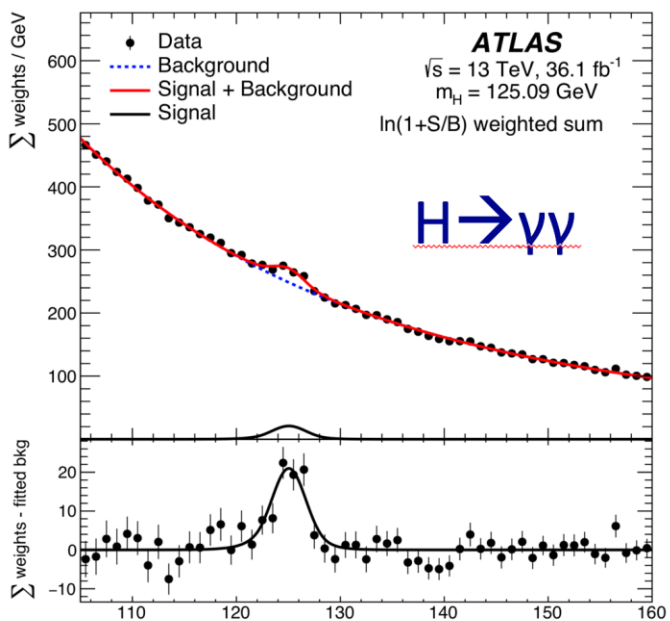
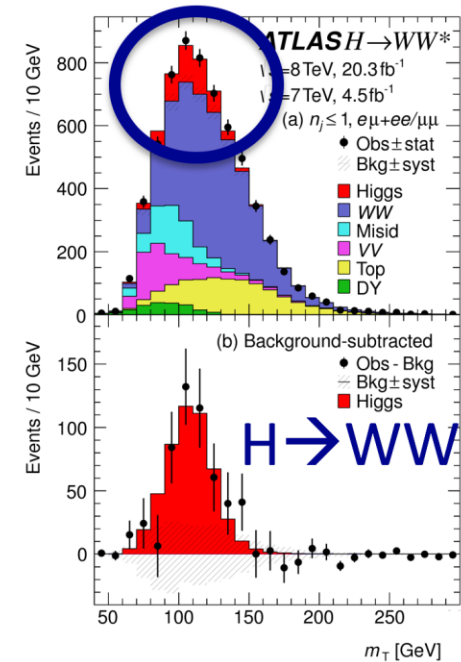
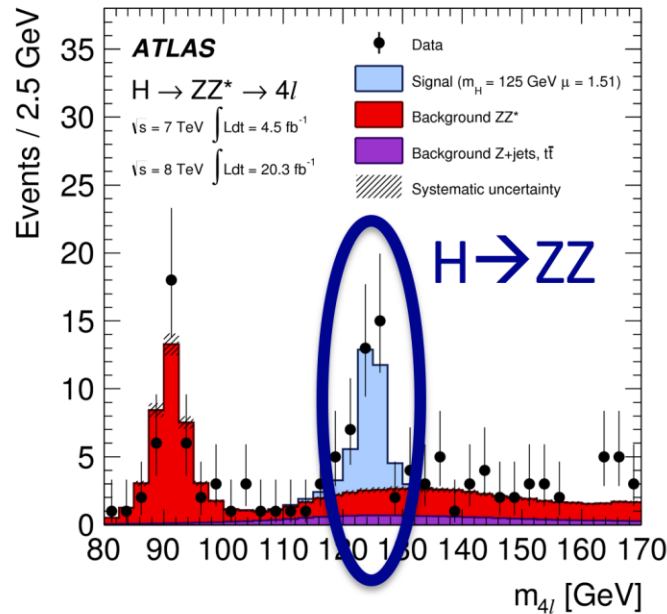
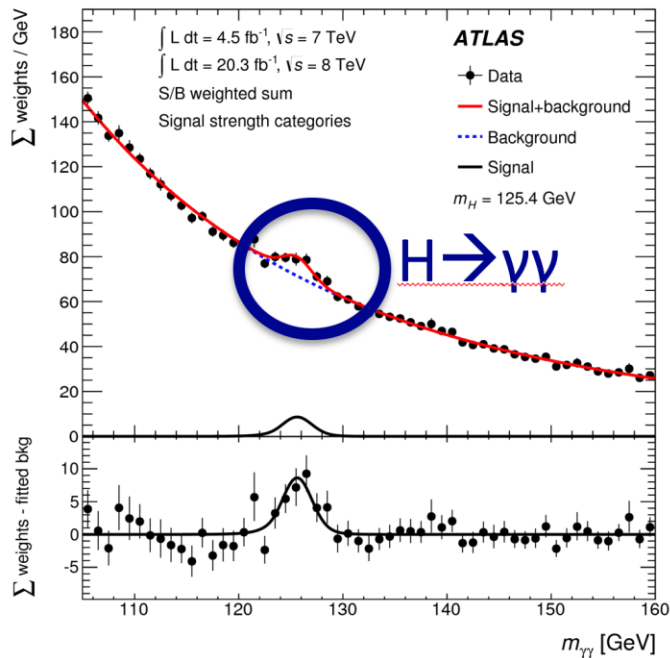


a lot of questions still open...

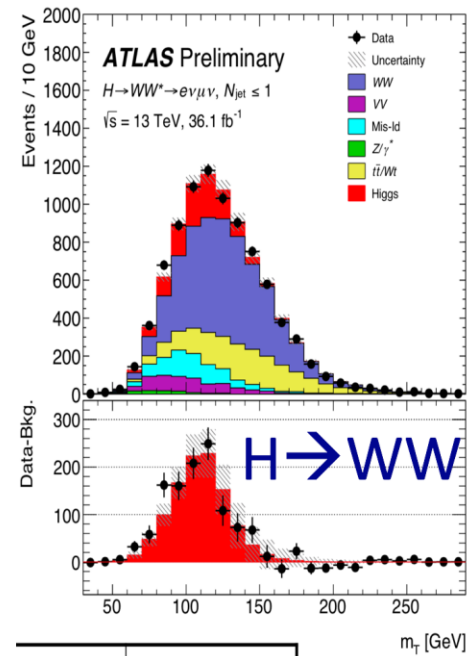
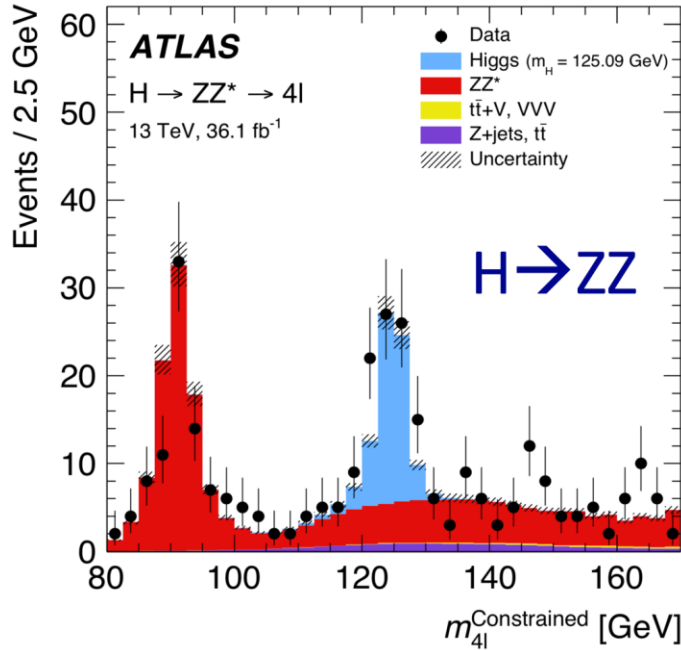
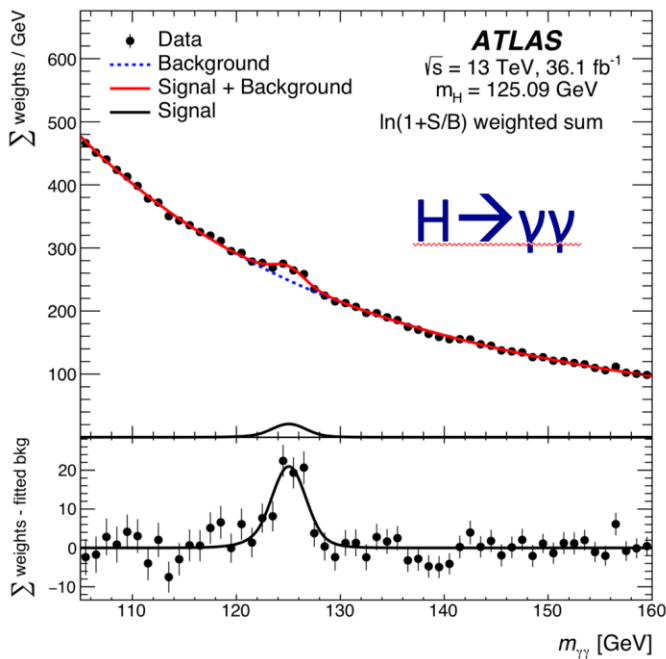
and today there's a few less questions thanks to ATLAS and not only, do you remember the discovery of Higgs boson?



do you remember the discovery of Higgs boson?



do you remember the discovery of Higgs boson?



A plot is not a *drawing* but is the synthesis of a work!!! and must communicate precisely the result obtained (**ROOT** helps to do this)

Let me start with **OS** again...

An operating system (OS) is system software that manages computer hardware and software resources and provides common services for computer programs.

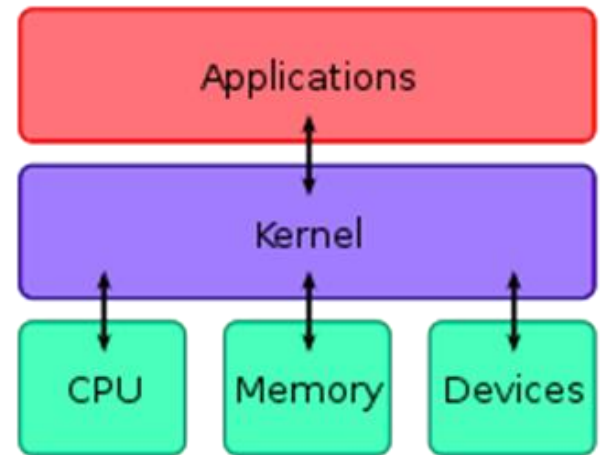
The components of an operating system all exist in order to make the different parts of a computer work together. All user software needs to go through the operating system in order to use any of the hardware, whether it be as simple as a mouse or keyboard or as complex as an Internet component.

Main components: **Kernel, I/O, networking, user interface, ...**

The kernel

The **kernel** provides the most basic level of control over all of the computer's hardware devices. It manages memory access for programs in the RAM, it determines which programs get access to which hardware resources, it sets up or resets the CPU's operating states for optimal operation at all times, and it organizes the data for long-term storage with file systems on such media as disks, tapes, flash memory, etc.

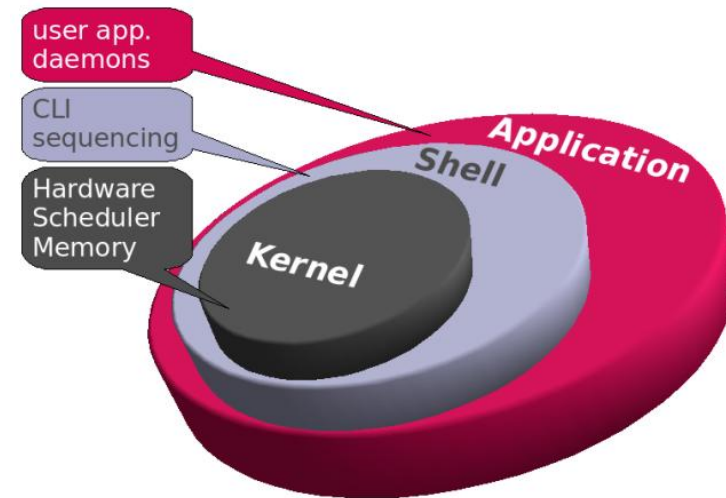
Among other things, a multiprogramming operating system kernel must be responsible for managing all system memory which is currently in use by the programs. This ensures that a program does not interfere with memory already in use by another program. Since programs time share, each program must have independent access to memory.



User interface

Every PC needs an interface for communication with us (someone speaks English, someone Italian, etc. and 'he' doesn't speak our language).

The user interface (named *shell*) views the directory structure and requests services from the operating system that will acquire data from input hardware devices, such as a keyboard, mouse etc., and requests operating system services to display prompts, status messages and such on output hardware devices, such as a video monitor or printer.



The two most common forms of a user interface have historically been the *command-line interface*, where computer commands are typed out line-by-line, and the *graphical user interface*, where a visual environment is present (icons and symbols).

When you open a window (**shell**) on the PC, you open a communication with your PC and you can talk to it...



let's try!

Now... we start!!!

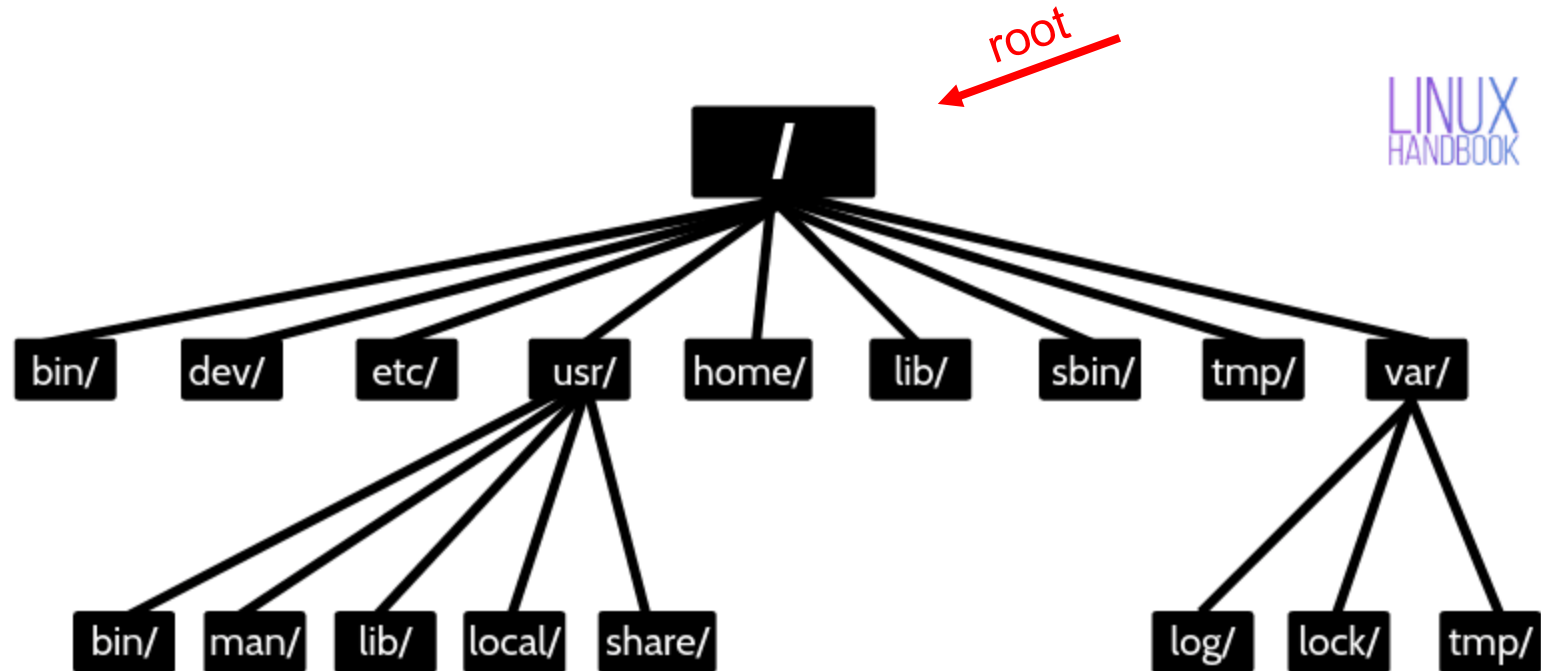
For convenience, we will use examples that you will find at this link (Copyright © Software Carpentry):

<https://swcarpentry.github.io/shell-novice/>

They kindly provide files with the only prescription you can find in:

https://swcarpentry.github.io/shell-novice/CODE_OF_CONDUCT.html

ubuntu directory structure

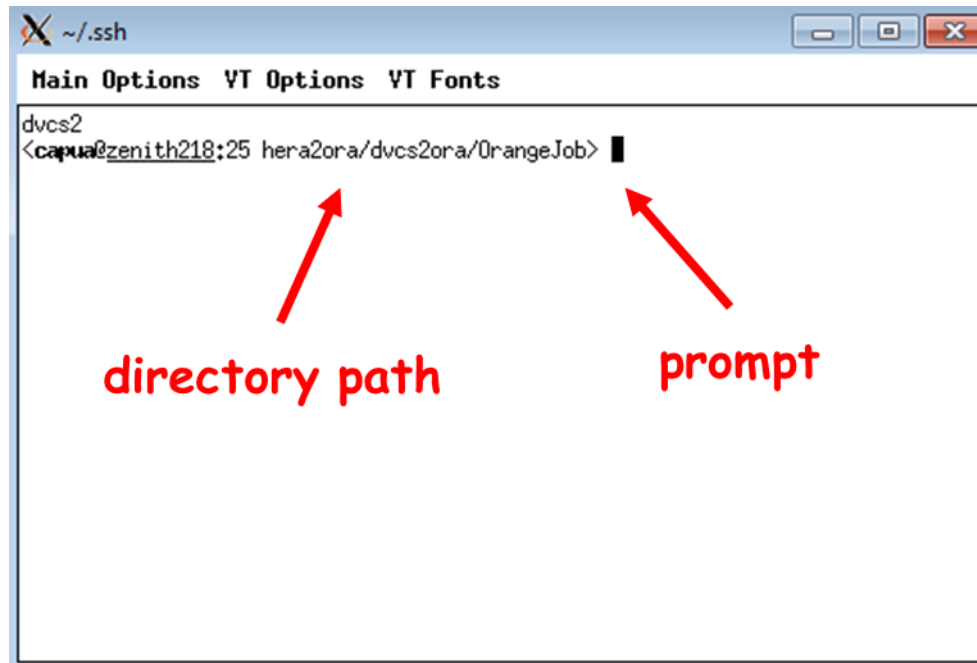


Note: <https://help.ubuntu.com/community/LinuxFilesystemTreeOverview>

First of all: *open a shell* (a terminal) and let's look:

you see a windows, a text, colors...

A *shell* is a sort of *communication port* (which, like a shell, hides the living part, the details). There are various shells: sh, csh, tcsh, zsh, bash...



Let's look...

type **ls** (means list directory contents)

type **man ls** (to quit press q) or type **ls --help** for help...

now **ls -l**

now **ls -la**

my preferred solution is **ls -latr** or I use sometimes **ls -lah**

Navigating Files and Directories

type **pwd** (which stands for 'print working directory')

which shows you the directory you are in

you can clear your terminal using the **clear** command

type **history** and see what happens.

now... exercise n.1

create a directory with **mkdir** command:

cd (why? **cd ..**)

mkdir lab2526

cd lab2526

mkdir shell-lesson-data

if you are in lab2526 type **ls** **she** and press tab ...

cd shell-lesson-data

pwd

questions:

- 1) How do I switch from shell-lesson-data to *my_home_dir* in one step?
- 2) Where am I if I type **cd** **/**? or only **cd**?

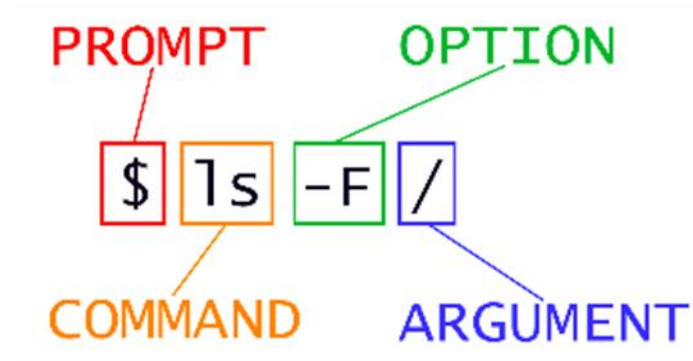
shortcuts:

The difference between `cd ..` and `cd -` is that the former brings you **up**, while the latter brings you **back**.

The shell interprets a tilde (~) character at the start of a path to mean “the current user’s home directory”. For example, you are in: `~/lab2526/shell-lesson-data/exercise-data`

type `cd ~` and then type `pwd`

now type `~/lab2526/shell-lesson-data` and then type `pwd`



A command is not a magic thing... please checked in /bin directory, you we see a long list of programs.../bin/ls or /bin/pwd or /bin/mkdir

the *root* rule /



Editors

There are various editors available: vim, emacs, nano, etc.
We will use **nano** at the beginning.

Esercise n.2

Create a directory with a valid name (no space, no -, no special characters): **mkdir xxx**

please, enter the dir: **cd xxx**

edit a new file: **nano test.txt**

write in: echo done!

Save and exit

now: type **bash test.txt**

A different way to create an empty file is:

touch test2 (filename extension can be removed but can be useful because it indicates which type of data the file holds and in some cases help a sw to understand a file: .pdf .png .mp4, etc.).

bash test2

ls -la test* and check privileges, dimension, etc.

then:

cp test2 test3

ls -la

more test3

mkdir old

mv test3 old/

ls (test3 is missing!)

ls old (it is there!)

mv test2 old/test22 (move and rename)

ls old

cp test.txt old/pippo.txt

ls old

cp -r old old2

ls

rm old (this doesn't work)

rm -r old (ok!)

ls

now... go into creatures directory

```
mkdir prova
```

```
cp basilisk.dat unicorn.dat minotaur.dat prova
```

```
ls prova
```

If given three file names, cp throws an error such as the one below, because it is expecting a directory name as the last argument.

Shorter solution (wildcard *):

```
cp *dat prova
```

Exercise n.3 (pipe |)

Please move in *shell-lesson-data/exercise-data/animal-counts* folder.

ls

more animals.csv

2012-11-05,deer,5

2012-11-05,rabbit,22

2012-11-05,raccoon,7

2012-11-06,rabbit,19

2012-11-06,deer,2

2012-11-06,fox,4

2012-11-07,rabbit,16

2012-11-07,bear,1

cat - concatenate files and print on the standard output, it is used to remove or ‘cut out’ certain sections of each line in the file. **cut** expects the lines to be separated into columns by a Tab character

```
$ cat animals.csv | head -n 5 | tail -n 3 | sort -r > final.txt
```

The **head** command extracts the first 5 lines from animals.csv. Then, the last 3 lines are extracted from the previous 5 by using the **tail** command. With the **sort -r** command those 3 lines are sorted in reverse order and finally, the output is redirected to a file final.txt.

The content of this file can be checked by executing **cat final.txt**

Check: build the pipeline up one command at a time to test your understanding.

wc -l, to show the number of lines in the file

wc command, to show only the number of characters or words in the files.

examples:

wc animals.csv

wc -l animals.csv

wc -l animals.csv > count.txt

wc -l animals.csv >> count.txt

The vertical bar (|) named **pipe**, between the two commands tells the shell that we want to use the output of the command on the left as the input to the command on the right:

sort -n count.txt | head -n 1

Loops are a programming construct which allow us to repeat a command or set of commands for each item in a list. I'll show you with some examples:

```
wc -l NENE*[AB].txt | sort -n | head -n 1  
sort -n NENE*.txt | head -n 1
```

```
for thing in list_of_things  
do  
    operation_using $thing # indentation is nice but not required  
done
```

```
$ for file in NENE*[AB].txt  
> do  
>     sort -n $file | head -n 1  
> done
```

```
$ for file in NENE*[AB].txt  
> do  
>     echo $file  
>     sort -n $file | head -n 1  
> done
```

Can you understand these two loops?

```
$ for loop_variable in 0 1 2 3 4 5 6 7 8 9  
> do  
>   echo $loop_variable  
> done
```

```
$ for datafile in *.pdb  
> do  
>   ls $datafile  
>   cat $datafile > test.pdb  
> done
```

Try to change datafile with filename...

```
$ for file in NENE*.txt  
> do  
> cp $file raw-$file  
> done
```

```
$ for file in NENE*.txt  
> do  
>   echo $file raw-$file  
>   cp $file raw-$file  
> done
```

```
$ for datafile in NENE*A.txt NENE*B.txt; do echo $datafile stats-$datafile;  
done
```

```
$ for datafile in NENE*A.txt NENE*B.txt; do bash goostats.sh $datafile stats-  
$datafile; done
```

```
$ for datafile in NENE*A.txt NENE*B.txt; do echo $datafile;  
bash goostats.sh $datafile stats-$datafile; done
```

```
#!/usr/bin/env bash
export A="hello world"
echo $A
```

When run as an executable using `./my-script.sh`, the `A` variable is not exported in your current shell and would just return an empty result.

```
$ ./test.sh
hello world
$ echo $A
```

When run the same script with the dot command using `. my-script.sh`, your current shell context will be changed. When to use the Dot Command in Bash?

The most obvious use case for using the dot command is when you want to change your current context by setting new variables or changing some existing one. You may also have a script that expect to change your current directory by using `cd`. It may also be convenient when trying to run a script that doesn't have the execute (x) permission.

In Bash, the source command is synonym to the dot command (`.`)